

1 SaaS Platform Architecture

Beyond algorithmic accuracy, the practical deployment of power grid inspection models demands a robust infrastructure capable of sustaining concurrent user requests, maintaining responsive interaction, and ensuring strict data isolation across organisational clients. To bridge the gap between static deep learning scripts and an industrial-grade service, we conceptualised and developed a fully functional Software-as-a-Service (SaaS) platform, detailed in this section.

1.1 Layered Architecture Design

The system adopts a highly decoupled four-tier architecture, separating visual presentation, application logic, inference orchestration, and data persistence to guarantee independent vertical scaling.

Figure 1: The four-tier layered architecture of the inspection SaaS platform.

- **Presentation Layer:** Implemented as a single-page HTML/JavaScript interactive dashboard, delivering real-time UI updates and communicating with the backend exclusively via asynchronous RESTful API endpoints.
- **Application Layer:** The Flask-based routing layer responsible for core Web concerns, including HTTP request handling, JWT/session authentication, payload validation, and tenant context resolution.
- **Service Layer:** The abstraction core that encapsulates background task orchestration, executes the event-driven queue, accesses the model registry, and performs GPU-accelerated inference, completely isolated from HTTP blocking concerns.
- **Data Layer:** Powered by SQLAlchemy ORM, managing the relational mapping of tenants, users, and historical inference tasks, enforcing row-level multi-tenant data isolation.

1.2 Asynchronous Task Processing Design

Synchronous inference mechanisms strictly bind an HTTP request to the inference function’s execution time. Under concurrent load, this design rapidly exhausts the finite Flask worker pool, triggering cascading HTTP 504 Gateway Timeouts.

To resolve this, the system implements an event-driven asynchronous workflow utilising a `ThreadPoolExecutor` with four constant memory workers—optimised to maximise GPU utilisation without provoking memory fault.

Figure 2: Asynchronous task submission, event-driven background processing, and client polling lifecycle.

The asynchronous state machine enforces a unidirectional lifecycle: `PENDING` → `RUNNING` → `SUCCESS` (or `FAILED`). When a client submits an inference payload, the Application

Layer immediately persists a **PENDING** metadata record and returns an HTTP 202 Accepted response containing a unique Task Identifier. Simultaneously, rather than implementing inefficient continuous database polling, the task context is passed via an in-memory queue to awaken an available background worker. The client subsequently queries the `/tasks/<id>/status` endpoint at 1.5-second polling intervals to retrieve progress and fetch the final Base64-encoded segmentation boundaries upon **SUCCESS**.

1.3 Multi-Tenant Data Isolation Design

Serving multiple distinct power grid subsidiaries on a shared physical infrastructure mandates strict cryptographic and logical boundaries to prevent unauthorised cross-tenant data leakage.

Figure 3: Entity-Relationship diagram demonstrating fundamental foreign-key constraints for tenant isolation.

The database schema relies on an interconnected entity hierarchy. The **Tenant** master table governs organisational metadata and API quota allocations. The **User** table acts as the credential entity, assigning Role-Based Access Control (RBAC) levels (Admin vs. Standard User) and bound by a strict `tenant_id` foreign key. Consecutively, every **InferenceTask** inherits both `tenant_id` and `user_id`.

To ensure absolute boundary sovereignty, isolation logic is structurally embedded at the Routing Layer. A Flask `before_request` interceptor securely extracts the cryptographic session token, validates it, and injects the `tenant_id` directly into the active request context. Consequently, every scoped SQLAlchemy database query is explicitly bound to this dynamic `tenant_id` filter condition prior to execution. This hard-coded predicate mechanism structurally guarantees that manually forged REST queries attempting to enumerate foreign Task IDs will securely default to a **404 Not Found** response.

1.4 Model Registry Design

Accommodating the heterogeneous memory signatures of distinct deep learning paradigms (e.g., Mask R-CNN, YOLOv8) within a uniformly constrained server environment (12 GB VRAM cap) necessitates intelligent resource scaling.

The platform employs a Software Design Adapter Pattern via a centralised **Model Registry**. This decouples the REST inference pipeline from the mathematical complexity of the underlying detectors.

Crucially, the registry enforces *Lazy-Loading* mechanics. During service initialisation, the system registers functional mappings for structural labels (e.g., `bird_nest`, `oil_leak`) but refrains from instantiating PyTorch weight tensors into GPU memory. Allocation is deferred strictly until the precise moment the first request for a specific algorithm is intercepted. This dramatically mitigates startup Out-of-Memory (OOM) crashes and prevents resource starvation. Future extensions scaling to supplementary grid defects (e.g., insulator cracking) require zero architectural revisions—developers simply append an independent mapping script to the static registry dictionary.